Portfolio Scorecard – Requirements

Source: Post-meeting notes * March 16, 2026 **File:** `portfolio-scorecard.html`

Summary

ID	Requirement	Status	Open Questions
REQ-01	Explainer subtitle on archetype section header	Ready to build	None
REQ-02	Custom archetype text input	Ready to build	2 (guidance fields, label copy)
REQ-03	Add Career Stage section (ported from UX Designer Scorecard)	Ready to build	Placement (subsection of 00 vs. own section)
REQ-04	Company career ladder toggle (Home Depot blueprint)	Ready to build	3 (extensibility, visual detail, attribution)
REQ-05	Career Goal / Next Step section	Ready to build	None (copy refinement is a skin concern)
REQ-06	Per-dimension notes (all rated areas)	Ready to build	None
REQ-07	Design Disciplines (multi-select + custom input)	Ready to build	None
REQ-08	Rebrand footer to “Jax UX” logo lockup	Ready to build	None
REQ-09	Replace Print with Download (client-side export)	Ready to build	2 (PNG vs PDF, keep print styles)
REQ-10	Shareable state via URL + QR code	Ready to build	None
REQ-11	Shell updates: rename Writing & Copy, add Storytelling dim	Ready to build	None
REQ-12	General Feedback section (bottom of form)	Ready to build	None
REQ-14	AI Tools & Fluency (Adaptability)	Ready to build	1 (broader adaptability scope – what else belongs here?)
REQ-15	Accessibility pass & typography scale	Ready to build (Phase 1 type scale, Phase 2 full WCAG audit)	None
REQ-16	Rename “Case Studies” -> “Project / Case Study”	Ready to build	None
REQ-13	Portfolio Purpose Framework & Leadership dims	Brainstorming (always last – requires scorecard version bump)	4 (scope, placement, lens visibility, standalone vs integrated)

REQ-01: Add Explainer Text to Section Header

Status: Ready to build **Priority:** – **Section affected:** Section 00 (Context) -> Designer archetype subsection

Current state

The archetype area has a single label:

> ***Designer archetype*** *(optional)*

No additional context is provided to help the user understand what archetype selection means or how to choose one.

Requirement

Add subtitle/helper copy beneath the “Designer archetype” label that explains the purpose of the selection. The explainer should communicate two ideas:

1. What part of design do you enjoy the most?
2. What are you best at?

This is **header-level subtitle text** – the same pattern used on the section header’s `section-subtitle` class – not per-pill descriptions or tooltips.

Reference (current pattern in codebase)

```
<!-- Existing section subtitle pattern -->
<div class="section-subtitle">
  Who are you, and what type of designer owns this portfolio?
</div>
```

The archetype explainer should follow this same visual hierarchy, sitting between the “Designer archetype” label and the pill row.

Suggested copy (refine as needed)

> *Select the archetype that best describes this designer – what part of design they enjoy most, or what they’re best at.*

REQ-02: Custom Archetype Input

Status: Ready to build **Priority:** – **Section affected:** Section 00 (Context) -> Designer archetype area (end of pill row)

Current state

The user can select from 7 predefined archetypes (Storyteller, Systems Architect, Brand Worldbuilder, Interaction Crafter, Service Designer, Research Strategist, Creative Technologist). If none fits, there is no alternative – the field is simply left empty.

Requirement

Add a custom input option **after** the existing archetype pills that allows the user to type their own archetype name if none of the 7 presets apply.

Behavior:

- Appears at the end of the archetype pill row (or immediately below it)
- Selecting a preset archetype should clear the custom input, and vice versa – they are mutually exclusive
- The custom archetype name should flow into the results panel wherever `designerName`’s archetype is displayed
- Custom input should be styled consistently with the existing pill/input aesthetic

Open questions

#	Question	Notes
1	Should a custom archetype include guidance fields (strength to show, watch for, case study emphasis)?	If yes, the results panel would need 3 additional text inputs. If no, the guidance section simply

		won’t appear for custom archetypes.
2	Should the section label change from “Designer archetype” to something friendlier?	Options: replace the label entirely with the new framing, or keep “Designer archetype” and add the explainer as helper text below it.

Engine impact

- `ARCHETYPES` array: No change to existing entries. Custom archetype would be stored as a separate state value (e.g., `customArchetype: string`).
- `PORTFOLIO_GUIDANCE` map: Would not have an entry for custom archetypes unless REQ-02 open question #1 is resolved as “yes.”
- Results panel: Needs a conditional – if custom archetype is set, display the name but skip or adapt the guidance section.

Suggested state changes

```
// New state
const [customArchetype, setCustomArchetype] = useState('');

// Derived
const activeArchetype = archetype || (customArchetype ? 'custom' : null);
const activeArchetypeName = archetype
  ? ARCHETYPES.find(a => a.id === archetype)?.name
  : customArchetype || null;
```

REQ-03: Add Career Stage Section

Status: Ready to build **Priority:** – **Section affected:** New section in the input form (before current Section 01 – Portfolio Shell) **Source:** Existing implementation in `ux-designer-scorecard.html`, Section 01

Requirement

Port the Career Stage selection from the UX Designer Scorecard into the Portfolio Scorecard. This is a single-select row of career level buttons, each displaying a title and a year-range badge.

Data model (from `ux-designer-scorecard.html`)

```
const CAREER_LEVELS = [
  { id: 'student', label: 'Student / Intern', yrs: '0 yrs' },
  { id: 'junior', label: 'Junior', yrs: '0-2 yrs' },
  { id: 'mid', label: 'Mid-Level', yrs: '2-5 yrs' },
  { id: 'senior', label: 'Senior', yrs: '5-8 yrs' },
  { id: 'lead', label: 'Lead', yrs: '7-10 yrs' },
  { id: 'principal', label: 'Principal / Staff', yrs: '10+ yrs' },
  { id: 'manager', label: 'Manager / Director', yrs: '8+ yrs' },
  { id: 'vp', label: 'VP / Head of Design', yrs: '12+ yrs' },
];
```

UI pattern (from `ux-designer-scorecard.html`)

- Horizontal row of pill-style buttons (`.ladder` / `.ladder-btn` classes)
- Each button shows the level label and a monospace year badge below it
- Single-select toggle: clicking the active level deselects it
- Selected state uses coral accent color in the designer scorecard; should adapt to the portfolio scorecard’s teal palette or follow whatever color the Context section uses

Placement

Currently the Portfolio Scorecard’s Section 00 (Context) contains review type and designer archetype. Career Stage should be added as a new subsection within Section 00, or promoted to its own numbered section. This shifts the existing section numbers:

Current	Proposed
00 – Context (review type + archetype)	00 – Context (review type + archetype + career stage)
01 – Portfolio Shell	01 – Portfolio Shell

02 – Case Studies	02 – Case Studies
03 – Portfolio Strategy	03 – Portfolio Strategy

Alternatively, if Career Stage gets its own section number, all subsequent sections shift by one.

State changes

```
// New state
const [careerLevel, setCareerLevel] = useState(null);

// Add to reset handler
setCareerLevel(null);

// Pass to results panel
const resultsProps = { ..., careerLevel };
```

Results panel impact

The UX Designer Scorecard displays the selected career level as a metadata field in the results sidebar. The same pattern should apply here – show the selected career stage as a contextual label in the Portfolio Assessment results panel.

REQ-04: Company Career Ladder Toggle (Home Depot Blueprint)

Status: Ready to build **Priority:** – **Section affected:** Career Stage section (REQ-03) **Depends on:** REQ-03 **Reference:** Home Depot UX Journey slide (uploaded image)

Requirement

Add a toggle within the Career Stage section that lets the user switch between two career ladder views:

1. **Generic ladder** – the current 8-level flat list from the UX Designer Scorecard (REQ-03)
2. **Home Depot ladder** – a company-specific dual-track path with IC and management branches

When the user toggles to the Home Depot view, the flat pill row is replaced with the branching career path from the blueprint.

Home Depot career path data model

The path has two entry points, a shared trunk, and then forks into two tracks with dashed crossover lines between them.

```
const HOMEDEPOT_CAREER_PATH = {
  id: 'homedepot',
  label: 'Home Depot',

  // Entry points (two starting roles)
  entry: [
    { id: 'intern', label: 'UX Intern' },
    { id: 'apprentice', label: 'UX Apprentice' },
  ],

  // Shared trunk (linear progression)
  trunk: [
    { id: 'designer_1', label: 'UX Designer I' },
    { id: 'designer_2', label: 'UX Designer II' },
    { id: 'senior', label: 'Senior UX Designer' },
    { id: 'staff', label: 'UX Staff Designer' },
  ],

  // Fork point: after Staff, path splits into IC and Management
  tracks: {
    ic: {
      label: 'Individual Contributor',
      levels: [
        { id: 'principal', label: 'UX Principal Designer' },
        { id: 'distinguished', label: 'Distinguished Designer' },
        { id: 'fellow', label: 'Design Fellow' },
      ],
    },
    management: {
      label: 'Management',
      levels: [
        { id: 'manager', label: 'UX Manager' },
        { id: 'sr_manager', label: 'Senior UX Manager' },
        { id: 'director', label: 'UX Director' },
        { id: 'sr_director', label: 'Sr. Director' },
      ],
    },
  },

  // Dashed crossover lines exist between IC and Management
  // at the Principal/Manager and Distinguished/Sr. Manager levels
  crossovers: [
    { from: 'principal', to: 'manager' },
    { from: 'distinguished', to: 'sr_manager' },
    { from: 'director', to: 'fellow' },
  ],
};
```

UI behavior

- **Toggle control:** A small toggle or segmented control above the ladder (e.g., “Generic” | “Home Depot”) lets the user switch views
- **Selection is preserved per-ladder:** If the user selects “Senior” on generic, switches to Home Depot and selects “UX Staff Designer,” then switches back, their generic selection should still be there. Only the active ladder’s selection is used in the results panel.
- **Home Depot view layout:** The branching path should visually communicate the dual-track fork – the flat pill row won’t work here. Options include a simple two-row layout after the fork point, or a mini flow diagram. The entry points (Intern / Apprentice) converge into the trunk, and after Staff the path splits top (IC) and bottom (Management).
- **Dashed crossover lines** from the blueprint are informational – they show that people can move between tracks. These can be rendered as visual indicators or simply omitted in v1 if layout complexity is too high.

State changes

```
// New state
const [ladderMode, setLadderMode] = useState('generic'); // 'generic' | 'homedepot'
const [careerLevel, setCareerLevel] = useState(null);      // generic selection
const [hdCareerLevel, setHdCareerLevel] = useState(null);   // home depot selection

// Derived: whichever ladder is active drives the results
const activeCareerLevel = ladderMode === 'generic' ? careerLevel : hdCareerLevel;
const activeCareerLabel = ladderMode === 'generic'
  ? CAREER_LEVELS.find(l => l.id === careerLevel)?.label
  : /* lookup from HOMEDEPOT_CAREER_PATH */;
```

Results panel impact

The results panel should display whichever career level is selected on the active ladder, plus a small indicator of which ladder is being used (e.g., “Senior UX Designer * Home Depot ladder”).

Open questions

#	Question	Notes
1	Should additional company ladders be supported in the future?	If yes, the toggle should be designed as a dropdown or extensible selector rather than a binary switch.
2	How visually detailed should the branching layout be?	Full flow diagram with arrows and crossover lines vs. simplified two-row layout.
3	Source attribution – should “Source: Home Depot” appear in the UI?	The original slide credits Home Depot. Consider a small attribution note.

REQ-05: Career Goal / Next Step Section

Status: Ready to build **Priority:** – **Section affected:** New subsection directly below Career Stage (REQ-03) **Depends on:** REQ-03, REQ-04

Requirement

Add a second career selection immediately below Career Stage that captures where the designer wants to go next. This gives the portfolio review a forward-looking dimension – the reviewer (or self-assessor) can evaluate whether the portfolio is positioned for the designer's goal, not just their current level.

Section header (refine copy as needed)

> ***Career Goal*** *Where do you want to grow – or what role are you targeting next?*

The framing should accommodate multiple intentions: moving one level up, staying at the same level but switching companies/domains, lateraling into management or back to IC, or simply deepening expertise in their current role.

UI behavior

- **Same ladder, same toggle:** This section mirrors whatever ladder mode the user selected in Career Stage (REQ-04). If they toggled to Home Depot, this section also shows Home Depot roles. The toggle lives in the Career Stage section and controls both.
- **Show all levels:** All levels are available for selection – not filtered by the Career Stage pick. A user at Senior might target a lateral move to a different Senior role, stay put, or aim for Lead. They might even select a level below their current one (career changers, people stepping back intentionally).
- **Same level is valid:** Selecting the same level as Career Stage is explicitly supported. This represents "I want to stay here and get better / find my next role at this level."
- **Single select, same toggle behavior:** Click to select, click again to deselect – same pattern as Career Stage.
- **Visual hint (optional):** If the user has selected both a current stage and a goal, a subtle visual indicator (e.g., a small arrow or "current -> goal" label) could connect the two selections. This is a nice-to-have, not a requirement.

State changes

```
// New state
const [careerGoal, setCareerGoal] = useState(null);       // generic goal
const [hdCareerGoal, setHdCareerGoal] = useState(null);    // home depot goal

// Derived: active goal follows ladder mode
const activeCareerGoal = ladderMode === 'generic' ? careerGoal : hdCareerGoal;

// Add to reset handler
setCareerGoal(null);
setHdCareerGoal(null);
```

Results panel impact

Display the career goal alongside the current career stage as a paired metadata field. Suggested format:

- **Current:** Senior UX Designer
- **Goal:** UX Staff Designer

Or as a single line: "Senior UX Designer -> UX Staff Designer"

If the user selects the same level for both, the results panel could reflect that with something like "Senior UX Designer * Deepening" or simply show the same label twice – this is a skin decision.

Relationship to portfolio evaluation

This pairing (current stage + goal) creates a lens for the portfolio review. A reviewer could ask: "Does this portfolio position the designer for their stated goal?" This is contextual metadata – it doesn't change scoring, but it enriches the guidance and pattern detection. Future iterations could use the gap between current and goal to surface targeted advice (e.g., "Your portfolio shows strong IC work but lacks the leadership narratives expected at the Manager level").

REQ-06: Per-Dimension Notes

Status: Ready to build **Priority:** – **Section affected:** All rated dimensions across all sections (Portfolio Shell, Case Studies, Portfolio Strategy)

Requirement

Every dimension that has a numeric rating (1-5 dots) should also have an optional note field. A small note icon appears on the card or row. Tapping it reveals a text input below the rating where the reviewer can leave qualitative feedback.

Affected areas

Section	Dimensions	UI pattern
01 – Portfolio Shell	8 dimensions	Cards in a 2-column grid
02 – Case Studies	6 dimensions x up to 6 case studies	Compact rows within each case study block
03 – Portfolio Strategy	5 dimensions	Cards in a 2-column grid

Total possible note fields: 8 + (6 x 6) + 5 = **49 max**

UI behavior

- **Note icon:** A small icon (e.g., pencil, chat bubble, or note glyph) on each card/row – subtle, not competing with the rating dots. Positioned near the rating or in the card's top-right area.
- **Toggle behavior:** Clicking the icon reveals a text area below the card or row. Clicking again (or clearing the text and blurring) hides it.
- **Visual indicator when note exists:** If a note has been written, the icon should show a filled/active state so the reviewer can scan and see which dimensions have notes at a glance.
- **Text area behavior (preferred):** Auto-resizing textarea that defaults to 1 row height and grows as the user types. Implemented via the `auto-resize` pattern:

```
// Auto-resize textarea
const handleNoteChange = (e) => {
  e.target.style.height = 'auto';
  e.target.style.height = e.target.scrollHeight + 'px';
  // update state...
};
```

```
.note-textarea {
  width: 100%;
  min-height: 32px;       /* ~1 row */
  resize: vertical;        /* fallback manual resize */
  overflow: hidden;         /* hide scrollbar during auto-resize */
  font-family: 'DM Sans', sans-serif;
  font-size: 12px;
  background: var(--input-bg);
  border: 1px solid var(--border);
  border-radius: 4px;
  padding: 8px 10px;
  color: var(--text);
}
```

- **Fallback:** If auto-resize proves unreliable, use a standard textarea with `resize: vertical` and a fixed starting height of ~1 row.

State changes

Notes are stored as parallel maps keyed by dimension ID, matching the existing score structures.

```
// New state
const [shellNotes, setShellNotes] = useState({});           // { first_impression: "text", ... }
const [strategyNotes, setStrategyNotes] = useState({});     // { variety: "text", ... }

// Case study notes live inside each case study object
// caseStudies: [{ id, name, scores, notes: { problem_framing: "text", ... } }]

// Add to reset handler
setShellNotes({});
setStrategyNotes({});
// Clear notes inside each case study on reset
```

Results panel impact

Notes are captured for export/print context but do not need to render in the live results sidebar – the sidebar is already dense. Notes should appear in the **print view** underneath each dimension's bar or score, so the printed scorecard includes the qualitative feedback alongside the ratings.

REQ-07: Design Disciplines Section

Status: Ready to build **Priority:** – **Section affected:** Section 00 (Context) – new subsection, separate from Designer Archetype

Concept distinction

This is a different concept from archetypes and should live as its own subsection:

- **Archetype** = design philosophy / identity ("How do you approach design?") -> single-select
- **Discipline** = craft specialization / practice area ("What kind of design work do you do?") -> multi-select

A designer could be a "Systems Architect" (archetype) who practices "Interaction Design" and "Service Design" (disciplines). The two selections complement each other and together paint a richer picture of who this portfolio belongs to.

Requirement

Add a multi-select discipline picker in the Context section. The user selects the craft areas that define their practice. A soft cap of 3-4 selections is encouraged via helper text (e.g., "Pick your top 3-4") but not strictly enforced in the UI.

Section header (refine copy as needed)

> ***Design Disciplines*** *What kind of design work defines your practice? Pick your top 3-4.*

Data model

```
const DESIGN_DISCIPLINES = [
  { id: 'visual', name: 'Visual Design' },
  { id: 'interaction', name: 'Interaction Design' },
  { id: 'service', name: 'Service Design' },
  { id: 'motion', name: 'Motion Design' },
  { id: 'research', name: 'UX Research' },
  { id: 'content', name: 'Content Design / UX Writing' },
  { id: 'systems', name: 'Design Systems' },
  { id: 'product', name: 'Product Design' },
  { id: 'brand', name: 'Brand Design' },
  { id: 'ia', name: 'Information Architecture' },
  { id: 'engineering', name: 'Design Engineering' },
  { id: 'accessibility', name: 'Accessibility / Inclusive Design' },
];
```

Custom discipline input

A free-text input at the end of the pill row (same pattern as REQ-02's custom archetype input) allows the user to type a discipline not in the list. Custom disciplines are added to the selection and count toward the soft cap.

UI behavior

- **Multi-select pills:** Same visual style as the archetype pills but with multi-select toggling – tapping a pill adds/removes it from the selection

- **Soft cap hint:** After 4 selections, the helper text could update subtly (e.g., shift to a muted color or add “You’ve selected N – consider narrowing to your strongest 3-4”) but all pills remain interactive
- **Custom input:** Appears at the end of the pill row. Typing and submitting adds a new pill to the selected set. Multiple custom entries are allowed.
- **Selected state styling:** Selected pills should use a consistent accent color (unlike archetypes where each has its own color). A single color keeps the multi-select visually clean.

State changes

```
// New state
const [disciplines, setDisciplines] = useState([]);          // ['visual', 'interaction', 'motion']
const [customDisciplines, setCustomDisciplines] = useState([]); // ['Design Ops', ...]

// Derived
const allSelectedDisciplines = [
  ...disciplines.map(id => DESIGN_DISCIPLINES.find(d => d.id === id)?.name),
  ...customDisciplines,
].filter(Boolean);

// Add to reset handler
setDisciplines([]);
setCustomDisciplines([]);
```

Results panel impact

Display selected disciplines as metadata tags in the results sidebar, near the archetype and career stage fields. Compact pill/tag display – no need for bars or scores since these aren’t rated.

Placement within Section 00

Suggested order within the Context section:

1. Review type (self / peer / mentor)
2. Designer archetype (REQ-01, REQ-02)
3. **Design disciplines (REQ-07)** <- new
4. Career stage (REQ-03, REQ-04)
5. Career goal (REQ-05)

Disciplines sit between archetype and career stage because the conceptual flow is: *who is reviewing -> what kind of designer -> what they practice -> where they are -> where they’re going.*

REQ-08: Rebrand Footer to “Jax UX” Logo Treatment

Status: Ready to build **Priority:** – **Section affected:** Footer (results panel) – `.footer-logo` element **Reference:** Uploaded Jax UX logo image

Current state

```
<div class="footer-logo">Jacksonville Design Community</div>
```

Styled as a single line of Bebas Neue at 16px with 2px letter-spacing.

Requirement

Replace “Jacksonville Design Community” with a stacked “JAX / UX” logo lockup that mirrors the uploaded brand image:

- **“JAX”** – smaller text, stacked on top
- **“UX”** – larger/bolder text below

Both lines should use Bebas Neue (already loaded) and be center-aligned. The size ratio from the reference image is roughly “JAX” at ~40% the size of “UX”. Suggested implementation:

```
<div class="footer-logo">
  <span class="logo-jax">JAX</span>
  <span class="logo-ux">UX</span>
</div>
```

```
.footer-logo {
  display: flex;
  flex-direction: column;
  align-items: center;
  line-height: 1;
}
.logo-jax {
  font-family: 'Bebas Neue', sans-serif;
  font-size: 12px;
  letter-spacing: 3px;
  color: var(--text-muted);
}
.logo-ux {
  font-family: 'Bebas Neue', sans-serif;
  font-size: 28px;
  letter-spacing: 2px;
  color: var(--text-muted);
}
```

Scope

This change applies to the footer in the results panel (sidebar and mobile). Check for any other instances of “Jacksonville Design Community” in the file and update consistently.

REQ-09: Replace Print Button with Download / Share

Status: Ready to build **Priority:** – **Section affected:** Results panel button row (`.rp-btn-row`)

Current state

```
<div className="rp-btn-row">
  <button className="btn btn-ghost" onClick={onReset}>Reset</button>
  <button className="btn btn-fill" onClick={() => window.print()}>Print</button>
</div>
```

The “Print” button triggers `window.print()`, which uses the existing `@media print` styles to render a printable view.

Requirement

Replace the Print button with a **Download** button that generates a downloadable file of the completed scorecard. The user can then attach this file to an email themselves.

Recommended approach: Client-side PDF or image generation

Since the scorecard is a standalone HTML file with no backend, the download must be generated entirely client-side. Two viable options:

Option A – HTML-to-image (recommended for simplicity)

Use `html2canvas` to capture the results panel as a PNG image.

```
import html2canvas from 'html2canvas';

const handleDownload = async () => {
  const el = document.querySelector('.results-panel');
  const canvas = await html2canvas(el);
  const link = document.createElement('a');
  link.download = `portfolio-scorecard-${designerName || 'assessment'}.png`;
  link.href = canvas.toDataURL();
  link.click();
};
```

CDN: `https://cdnjs.cloudflare.com/ajax/libs/html2canvas/1.4.1/html2canvas.min.js`

Option B – HTML-to-PDF (richer output, heavier dependency)

Use `html2pdf.js` (wraps html2canvas + jsPDF) to generate a multi-page PDF.

```
import html2pdf from 'html2pdf.js';

const handleDownload = () => {
  const el = document.querySelector('.results-panel');
  html2pdf().set({
    margin: 16,
    filename: `portfolio-scorecard-${designerName || 'assessment'}.pdf`,
    image: { type: 'png', quality: 0.95 },
    html2canvas: { scale: 2 },
    jsPDF: { unit: 'mm', format: 'a4', orientation: 'portrait' },
  }).from(el).save();
};
```

CDN: `https://cdnjs.cloudflare.com/ajax/libs/html2pdf.js/0.10.1/html2pdf.bundle.min.js`

What to capture in the download

The downloaded file should include the full results panel content: overall score ring, pattern diagnosis, section scores, bar charts, case study ranking, growth areas, archetype guidance, and any notes (REQ-06) if they exist. The designer name, reviewer name, date, career stage, career goal, and selected disciplines should all be visible as metadata in the output.

UI changes

- Button label: **“Download”** or **“Export”** (with a small download icon)
- Button style: Keep the current `btn-fill` style (teal accent)
- On click: Generate the file and trigger browser download – no modal, no email prompt
- Loading state: Show a brief spinner or “Generating…” text while the canvas renders, since html2canvas can take a moment on complex DOM

Button row layout (updated)

```
<div className="rp-btn-row">
  <button className="btn btn-ghost" onClick={onReset}>Reset</button>
  <button className="btn btn-fill" onClick={handleDownload}>Download</button>
</div>
```

Open questions

#	Question	Notes
1	PNG or PDF?	PNG is simpler and lighter. PDF is more professional and paginated. Could offer both via a small dropdown on the button.
2	Should the existing print styles be kept?	Users can still Cmd+P / Ctrl+P manually. Removing `@media print` styles would simplify the CSS but lose a free feature. Recommend keeping them.

REQ-10: Shareable State via URL + QR Code

Status: Ready to build **Priority:** – **Section affected:** Global (state management), Results panel (new Share button / QR display) **Dependencies:** None (zero external services required)

Requirement

Encode the scorecard's scores and selections into the URL so a saved link (or QR code scan) restores the full evaluation state. This allows reviewers to save, share, or revisit their assessment without a backend, login, or database.

What gets encoded

Included (scores + selections only):

Data	Type	Est. size
`designerName`	string	~20 chars
`reviewerName`	string	~20 chars
`reviewerType`	enum	4-6 chars
`archetype`	enum or null	~12 chars
`customArchetype`	string	~20 chars
`disciplines`	array of IDs	~40 chars
`customDisciplines`	array of strings	~40 chars
`ladderMode`	enum	~8 chars
`careerLevel` / `hdCareerLevel`	enum	~12 chars
`careerGoal` / `hdCareerGoal`	enum	~12 chars
`shellScores`	8 x digit	~24 chars
`caseStudies` (names + scores)	6 x (name + 6 digits)	~180 chars
`strategyScores`	5 x digit	~15 chars
Estimated total (pre-encoding)		**~400 chars**

Excluded:

Data	Reason
Notes (REQ-06)	Free text is too large – a few sentences per field x 49 fields would blow past URL and QR limits
Date	Auto-generated on load, not user input

Encoding strategy

```
// Serialize state to a compact JSON object
const stateToShare = {
  dn: designerName,
  rn: reviewerName,
  rt: reviewerType,
  ar: archetype,
  ca: customArchetype,
  di: disciplines,
  cd: customDisciplines,
  lm: ladderMode,
  cl: careerLevel,
  hcl: hdCareerLevel,
  cg: careerGoal,
  hcg: hdCareerGoal,
  sh: shellScores,
  cs: caseStudies.map(c => ({ i: c.id, n: c.name, s: c.scores })),
  st: strategyScores,
};

// Encode: JSON -> compress -> base64 -> URL hash
const encoded = btoa(JSON.stringify(stateToShare));
const shareUrl = `${window.location.origin}${window.location.pathname}#s=${encoded}`;

// Decode on load
useEffect(() => {
  const hash = window.location.hash;
  if (hash.startsWith('#s=')) {
    try {
      const decoded = JSON.parse(atob(hash.slice(3)));
      // Hydrate state from decoded object...
    } catch (e) {
      console.warn('Invalid share URL');
    }
  }
}, []);
```

Estimated encoded URL length: ~600-800 characters (well within the ~2,000 safe limit for browsers, email clients, and messaging apps).

QR code generation

Use a client-side QR library to render the share URL as a scannable QR code. At ~600-800 characters, the QR code stays at a comfortable density level (Version ~15-20, easily scannable by any phone camera).

```
// Using qrcode.js (lightweight, no dependencies)
// CDN: https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js

const generateQR = (url) => {
  new QRCode(document.getElementById('qr-container'), {
    text: url,
    width: 160,
    height: 160,
    colorDark: '#1a1a2e',   // or adapt to current theme
    colorLight: '#ffffff',
  });
};
```

UI behavior

- **Share button:** Add a “Share” button to the results panel button row (alongside Reset and Download). Clicking it:
  1. Serializes current state into the URL hash
  2. Copies the URL to clipboard with a brief “Copied!” confirmation
  3. Displays a QR code below the button row (or in a small modal/popover)
- **Auto-restore on load:** If the URL contains a `#s=` hash on page load, the app hydrates all state from it before the user interacts with anything
- **URL updates live (optional):** As the user fills out the scorecard, the URL hash could update in real-time via `history.replaceState()` so the URL is always current. This is a nice-to-have – it means the user can bookmark or copy the URL at any point without hitting “Share” first.

Button row layout (updated)

```
<div class="rp-btn-row">
  <button className="btn btn-ghost" onClick={onReset}>Reset</button>
  <button class="btn btn-ghost" onClick={handleShare}>Share</button>
  <button class="btn btn-fill" onClick={handleDownload}>Download</button>
</div>
```

Cost

$0. Everything is client-side: `btoa` / `atob` are built-in, `qrcode.js` is a free CDN library (~4KB), no API keys, no backend, no rate limits, works offline.

REQ-11: Portfolio Shell Dimension Updates

Status: Ready to build **Priority:** – **Section affected:** Section 01 – Portfolio Shell (`SHELL_DIMS` array)

11a: Rename "Writing & Copy" -> "Communication: Writing & Copy"

Current:

```
{ id: 'writing_quality', name: 'Writing & Copy', desc: 'Clear, concise, confident, error-free', icon: '', ... }
```

Updated:

```
{ id: 'writing_quality', name: 'Communication: Writing & Copy', desc: 'Clear, concise, confident, error-free', icon: '', ..
```

The `id` stays the same (`writing_quality`) so existing URL-encoded state (REQ-10) remains compatible. Only the display `name` changes.

11b: Add new "Storytelling" dimension

Add a new dimension to the Portfolio Shell that evaluates how well the portfolio site itself tells a story – the narrative thread that connects the homepage through case studies to the CTA. This is distinct from the per-case-study "Narrative & Flow" dimension in Section 02, which evaluates storytelling within individual case studies.

```
{
  id: 'storytelling',
  name: 'Storytelling',
  desc: 'The portfolio tells a cohesive story from landing page through case studies',
  icon: '',   // or , -- skin decision
  color: '#f472b6',
  glow: 'rgba(244,114,182,0.12)',
}
```

Placement within SHELL_DIMS

Suggested insertion after "About & Personal Brand" since storytelling is closely related to personal narrative:

#	Current	Proposed
01	First Impression	First Impression
02	Navigation & Structure	Navigation & Structure
03	About & Personal Brand	About & Personal Brand
04	Visual Design Quality	**Storytelling** <- new
05	Responsiveness & A11y	Visual Design Quality
06	Writing & Copy	Responsiveness & A11y
07	Contact & CTA	Communication: Writing & Copy <- renamed
08	Performance & Polish	Contact & CTA
09	–	Performance & Polish

Scoring impact

Shell section max increases from **40** (8 x 5) to **45** (9 x 5). This affects:

- `grandMax` – increases from 95 to 100 (a cleaner number)
- Progress ring percentage calculation
- Pattern detection thresholds in `detectPattern()` – the shell thresholds (e.g., `shellTotal >= 34` for “Portfolio Ready”) need to be proportionally adjusted for the new max
- Results panel stat display (`shellTotal / 40` -> `shellTotal / 45`)

Updated pattern detection thresholds (suggested)

Pattern	Current shell threshold	Adjusted (9 dims)
Portfolio Ready	>= 34 / 40 (85%)	>= 38 / 45 (84%)
Strong Foundation	>= 28 / 40 (70%)	>= 32 / 45 (71%)
All Sizzle, No Steak	>= 32 / 40 (80%)	>= 36 / 45 (80%)
Buried Treasure	<= 24 / 40 (60%)	<= 27 / 45 (60%)

REQ-12: General Feedback Section

Status: Ready to build **Priority:** – **Section affected:** New section at the bottom of the main input form, after Section 03 (Portfolio Strategy)

Requirement

Add a final open-ended text area where the reviewer can write overall thoughts, impressions, and recommendations about the portfolio. This is the “big picture” counterpart to the per-dimension notes (REQ-06) – a place for the reviewer to synthesize everything into a holistic takeaway.

Section header

```
04 -- General Feedback
Your overall thoughts, impressions, and recommendations
```

Section number follows Portfolio Strategy (currently 03). The accent color should feel like a conclusion – could use the teal from the header or a neutral tone to signal “wrap-up.”

UI

- **Single large textarea** – no cards, no dots, no rating. Just a generous text input.
- **Auto-resize behavior:** Same pattern as REQ-06 notes – starts at ~3 rows and grows as the user types. Falls back to `resize: vertical` if auto-resize is unreliable.
- **Placeholder copy (refine as needed):** “What stood out? What’s the single biggest thing this designer should focus on next?”
- **Character count (optional):** A subtle character count in the corner could help reviewers gauge length, but is not required.

```
.feedback-textarea {
  width: 100%;
  min-height: 80px;          /* ~3 rows */
  resize: vertical;
  font-family: 'DM Sans', sans-serif;
  font-size: 13px;
  line-height: 1.6;
  background: var(--input-bg);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 14px 16px;
  color: var(--text);
}
.feedback-textarea:focus {
  border-color: var(--teal);
  box-shadow: 0 0 0 3px var(--teal-glow);
}
```

State changes

```
// New state
const [generalFeedback, setGeneralFeedback] = useState('');

// Add to reset handler
setGeneralFeedback('');
```

Results panel / export impact

- **Results sidebar:** Does not display the feedback text (sidebar is for scores and diagnostics, not prose).
- **Download (REQ-09):** The general feedback should appear at the bottom of the exported PDF/PNG, below all scores and diagnostics.
- **Print view:** Same – feedback appears as the final section in the printed output.
- **URL state (REQ-10):** Excluded from URL encoding (same rationale as notes – free text is too large for URL/QR).

REQ-14: AI Tools & Fluency (Adaptability)

Status: Ready to build **Priority:** – **Section affected:** New section in the input form – suggested placement after Portfolio Strategy (Section 03) and before General Feedback (REQ-12) **Type:** Checklist / multi-select (similar to Professional Assets in the UX Designer Scorecard)

Requirement

Add a section where the designer (or reviewer) can indicate which AI tools they have exposure to. AI fluency is becoming a real differentiator in portfolio evaluation – not just "do they use AI" but *which tools* and *how deeply* they've integrated them into their workflow.

This is not a rated/scored section (no 1-5 dots). It's a checklist that captures tool exposure, organized by category, with status indicators similar to the Professional Assets pattern in the UX Designer Scorecard.

Open question: Adaptability scope

The "Adaptability" framing opens the door to more than just AI tools. Other categories that could live under this umbrella include:

- **Design Tool Range** – Figma, Sketch, Adobe CC, Framer, Webflow, etc.
- **Platform Diversity** – web, native mobile, tablet, kiosk, wearable, voice, spatial/AR
- **Methodology Fluency** – agile, design thinking, lean UX, jobs-to-be-done, double diamond
- **No-Code / Low-Code Tools** – Webflow, Framer, Notion, Airtable, Zapier
- **Emerging Tech Exposure** – spatial computing, voice UI, AR/VR, conversational design

These need a follow-up discussion to decide: which belong here, whether they're separate subsections or a mixed checklist, and whether any overlap with existing dimensions (e.g., Platform Diversity vs. Strategy > Range & Variety).

Suggested status options per tool

```
const AI_TOOL_STATUSES = [
  { id: 'none', label: 'No exposure' },
  { id: 'exploring', label: 'Exploring' },
  { id: 'using', label: 'Using regularly' },
  { id: 'advanced', label: 'Advanced / Teaching others' },
];
```

Data model

Tools organized by category, with MCP integrations and SaaS clients highlighted.

```
const AI_TOOLS = [
  // --- AI Code Editors & Dev Tools ---
  { id: 'cursor', name: 'Cursor', category: 'AI Code Editors', desc: 'AI-powered code editor built on VS Code' },
  { id: 'cursor_rules', name: 'Cursor Rules', category: 'AI Code Editors', desc: 'Custom rule files for guiding Cursor beha
  { id: 'cursor_cli', name: 'Cursor CLI', category: 'AI Code Editors', desc: 'Command-line interface for Cursor workflows'
  { id: 'windsurf', name: 'Windsurf', category: 'AI Code Editors', desc: 'AI code editor with agentic workflows' },
  { id: 'github_copilot', name: 'GitHub Copilot', category: 'AI Code Editors', desc: 'AI pair programmer in VS Code / JetBr
  { id: 'claude_code', name: 'Claude Code', category: 'AI Code Editors', desc: 'Anthropic CLI tool for agentic coding' },

  // --- MCP Integrations ---
  { id: 'figma_mcp', name: 'Figma MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'Design-to-code bridge via Model Co
  { id: 'talk_to_figma', name: 'TalkToFigma MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'Agentic Figma reading an
  { id: 'notion_mcp', name: 'Notion MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'AI access to Notion docs and dat
  { id: 'slack_mcp', name: 'Slack MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'AI-driven Slack search and messagi
  { id: 'google_drive_mcp', name: 'Google Drive MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'AI access to Drive f
  { id: 'github_mcp', name: 'GitHub MCP', category: 'MCP Integrations', tag: 'MCP', desc: 'AI access to repos, issues, PRs'

  // --- AI Design & Prototyping SaaS ---
  { id: 'figma_ai', name: 'Figma AI / Figma Make', category: 'AI Design SaaS', tag: 'SaaS', desc: 'Native AI features -- fi
  { id: 'v0', name: 'v0 (Vercel)', category: 'AI Design SaaS', tag: 'SaaS', desc: 'Text-to-UI component generation with sha
  { id: 'lovable', name: 'Lovable', category: 'AI Design SaaS', tag: 'SaaS', desc: 'Text-to-full-app builder for MVPs and p
  { id: 'bolt', name: 'Bolt', category: 'AI Design SaaS', tag: 'SaaS', desc: 'Full-stack AI builder with visual + code view
  { id: 'replit', name: 'Replit Agent', category: 'AI Design SaaS', tag: 'SaaS', desc: 'AI-assisted app building in the bro
  { id: 'relume', name: 'Relume', category: 'AI Design SaaS', tag: 'SaaS', desc: 'AI wireframe and sitemap generation' },

  // --- AI Image & Visual Generation ---
  { id: 'midjourney', name: 'Midjourney', category: 'AI Visual', desc: 'Image generation for moodboards, concepts, explorat
  { id: 'dalle', name: 'DALL*E / ChatGPT image gen', category: 'AI Visual', desc: 'OpenAI image generation' },
  { id: 'firefly', name: 'Adobe Firefly', category: 'AI Visual', desc: 'Commercially safe AI image generation' },
  { id: 'adobe_ai', name: 'Adobe Sensei / Creative Cloud AI', category: 'AI Visual', desc: 'AI features in Photoshop, Illus

  // --- AI Research & Content ---
  { id: 'chatgpt', name: 'ChatGPT', category: 'AI Assistants', desc: 'General-purpose AI for research, writing, brainstormi
  { id: 'claude', name: 'Claude', category: 'AI Assistants', desc: 'Anthropic AI for analysis, writing, coding' },
  { id: 'gemini', name: 'Gemini', category: 'AI Assistants', desc: 'Google AI assistant' },
  { id: 'perplexity', name: 'Perplexity', category: 'AI Assistants', desc: 'AI-powered research and search' },
  { id: 'notion_ai', name: 'Notion AI', category: 'AI Assistants', tag: 'SaaS', desc: 'AI writing and summarization in Noti

  // --- AI UX-Specific ---
  { id: 'maze', name: 'Maze AI', category: 'AI UX Tools', tag: 'SaaS', desc: 'AI-powered usability testing and research ana
  { id: 'ux_pilot', name: 'UX Pilot', category: 'AI UX Tools', tag: 'SaaS', desc: 'AI rapid concept screen generation' },
  { id: 'uizard', name: 'Uizard', category: 'AI UX Tools', tag: 'SaaS', desc: 'AI wireframing and design generation' },
  { id: 'frontitude', name: 'Frontitude', category: 'AI UX Tools', tag: 'SaaS', desc: 'AI UX copy and content consistency'
  { id: 'magician', name: 'Magician (Figma plugin)', category: 'AI UX Tools', desc: 'AI copy, icons, and image generation i

  // --- AI Motion & Video ---
  { id: 'runway', name: 'Runway', category: 'AI Motion', desc: 'AI video generation and editing' },
  { id: 'kling', name: 'Kling', category: 'AI Motion', desc: 'AI video generation' },
  { id: 'lottie_ai', name: 'LottieFiles AI', category: 'AI Motion', desc: 'AI-generated animations and micro-interactions'
];
```

UI pattern

Group tools by category with collapsible category headers. Each tool row shows the tool name, a brief description, an optional tag badge (MCP / SaaS), and the status selector.

```
+- AI Code Editors ---------------------------------+
|  Cursor                          [Exploring v]     |
|  AI-powered code editor                            |
|                                                    |
|  Cursor Rules                    [No exposure v]   |
|  Custom rule files for guiding behavior            |
|  ...                                               |
+- MCP Integrations ------------------- [MCP] -------+
|  Figma MCP                       [Using regularly] |
|  Design-to-code bridge via MCP                     |
|  ...                                               |
+----------------------------------------------------+
```

Custom tool input

Same pattern as REQ-02 and REQ-07 – a free-text input at the bottom of the section for tools not in the list. The user enters a name and selects a status.

State changes

```
// New state
const [aiToolStatuses, setAiToolStatuses] = useState({});
// { cursor: 'using', figma_mcp: 'advanced', ... }

const [customAiTools, setCustomAiTools] = useState([]);
// [{ name: 'Framer AI', status: 'exploring' }, ...]

// Add to reset handler
setAiToolStatuses({});
setCustomAiTools([]);
```

Results panel impact

Display a summary in the results sidebar: total tools with exposure, breakdown by category, and highlight any “Advanced” items. Compact format – not a full list, just a quick signal of AI fluency depth.

Suggested format:

```
AI FLUENCY
12 tools * 3 advanced
MCP: 2 * SaaS: 5 * Code: 3 * Visual: 2
```

URL state (REQ-10)

AI tool statuses can be encoded – they’re just an object of short ID -> enum mappings. Estimated ~100-200 additional characters. Custom tools add more but should stay manageable.

REQ-15: Accessibility Pass & Typography Scale

Status: Ready to build **Priority:** High – users are reporting readability issues **Section affected:** Global (entire scorecard) **Trigger:** Multiple people saying the tool is hard to read. Treat this as a smell – the real issue is accessibility hasn’t been systematically addressed.

The font size problem

The scorecard currently has **30+ elements below 14px**, some as low as **7px**. Here’s the full audit:

Critical violations (below 10px – nearly unreadable)

Size	Element	Class
7px	Stat labels in results	`.rp-stat-label`
8px	Rating scale labels (“Weak” / “Strong”)	`.rating-label`
8px	Section title labels in results	`.rp-section-title`
8px	Guidance labels	`.guidance-label`
8px	Footer sub text	`.footer-sub`
8px	Case study dot numbers	`.cs-rating .dot-num`
9px	Bar values in results	`.rp-bar-val`
9px	Case study rank scores	`.cs-rank-score`
9px	Case study tier labels	`.cs-rank-tier`
9px	Growth area scores	`.rp-growth-score`
9px	Stat max values	`.rp-stat-max`
9px	Results panel tag	`.rp-tag`
9px	Progress ring label	`.progress-ring-label`
9px	Button text	`.btn`
9px	Dot numbers	`.dot-num`

Sub-14px violations

Size	Element	Class
10px	Tag badges	`.tag`
10px	Meta labels	`.meta-label`
10px	Date display	`.date-display`
10px	Bar labels in results	`.rp-bar-label`
10px	Case study dim descriptions	`.cs-dim-desc`
11px	Archetype pills	`.arch-pill`
11px	Card descriptions	`.card-desc`
11px	Case study rank names	`.cs-rank-name`
11px	Pattern descriptions	`.pattern-desc`
11px	Growth area names	`.rp-growth-name`
11px	Guidance text	`.guidance-text`
11px	Theme toggle icon	`.theme-toggle .icon`
12px	Section subtitles	`.section-subtitle`
12px	Reviewer type buttons	`.rev-btn`
12px	Case study dim names	`.cs-dim-name`
13px	Page subtitle	`.subtitle`
13px	Add case study button	`.add-cs-btn`
13px	Pattern placeholder	`.pattern-placeholder`

Already at or above 14px (compliant)

Size	Element	Class
14px	Meta inputs	`.meta-input`
14px	Card names	`.card-name`
14px	Remove button	`.cs-remove`
15px	Case study name input	`.cs-name-input`
16px	Footer logo	`.footer-logo`
20px+	Section numbers, headings, stat numbers	various display type

Requirement

Establish a proper typographic scale based on WCAG best practices and readability standards. Not a blanket minimum – a tiered approach that matches text size to its role.

Recommended type scale (accessibility-informed)

Role	Recommended size	Rationale
Body text, descriptions, helper text	16px	Browser default. WCAG best practice for primary readable content.
Interactive labels (buttons, pills, inputs)	14-15px	Must be legible at a glance during interaction.
Secondary metadata (tags, timestamps, stat annotations, scale labels)	12-13px	Acceptable for non-essential context if contrast ratio meets AA (4.5:1).
Decorative chrome (dot numbers inside circles, watermark card numbers, footer fine print)	10-11px	Redundant with other visual signals – not required for comprehension.
Absolute floor	**12px**	Nothing that carries meaning goes below

		12px. Period.

What changes (applying the scale to the current audit)

Must increase (currently carrying meaning at 7-9px)

Element	Current	Proposed	Why
`.rp-stat-label`	7px	12px	Labels on score stats – users need to read these
`.rating-label` (“Weak”/“Strong”)	8px	12px	Scale anchors – essential for understanding ratings
`.rp-section-title`	8px	13px	Section headers in results – organizational text
`.guidance-label`	8px	12px	Labels for guidance cards
`.dot-num`	9px	10px	Decorative – redundant with dot position, but slightly larger helps
`.cs-rating .dot-num`	8px	10px	Same – decorative exception
`.rp-bar-val`	9px	12px	Score values – users read these to understand results
`.cs-rank-score`	9px	12px	Score display – essential
`.rp-growth-score`	9px	12px	Score display – essential
`.rp-stat-max`	9px	12px	“/40” denominators – contextual but meaningful
`.btn`	9px	14px	Buttons – primary interactive elements
`.rp-tag`	9px	12px	Section identifier
`.progress-ring-label`	9px	12px	Score context label

Should increase (currently 10-11px, below body standard)

Element	Current	Proposed	Why
`.tag`	10px	12px	Badge label
`.meta-label`	10px	12px	Form labels
`.date-display`	10px	12px	Metadata
`.rp-bar-label`	10px	13px	Dimension names in results – users scan these
`.cs-dim-desc`	10px	14px	Descriptions users read to understand what they’re rating
`.card-desc`	11px	14px	Same – rating guidance text
`.arch-pill`	11px	14px	Interactive selection text
`.cs-rank-name`	11px	14px	Case study names in results

`.pattern-desc`	11px	14px	Pattern diagnosis explanation
`.guidance-text`	11px	14px	Guidance content – meant to be read carefully
`.rp-growth-name`	11px	14px	Growth area names
`.section-subtitle`	12px	14px	Section descriptions
`.rev-btn`	12px	14px	Interactive button text
`.cs-dim-name`	12px	14px	Dimension names users read while rating

Fine as-is

Element	Current	Rationale
`.meta-input`	14px	Compliant
`.card-name`	14px	Compliant
`.cs-name-input`	15px	Compliant
Section numbers, headings, stat display numbers	20-68px	Display type – no issue
`.footer-sub`	8px -> 10px	Decorative fine print, but bump slightly
`.theme-toggle .icon`	11px	Decorative icon glyphs – acceptable

Broader accessibility audit

The font size complaint is a **smell** – it signals that accessibility hasn't been systematically addressed. A full accessibility pass should cover:

Color & contrast: - Audit all `--text-dim` and `--text-muted` values against their backgrounds in both light and dark themes - Ensure all text meets WCAG 2.1 AA contrast ratios (4.5:1 for normal text, 3:1 for large text) - Check that color is never the *only* indicator of state (rated vs. unrated, selected vs. unselected)

Keyboard navigation: - All interactive elements (dots, pills, buttons, toggles) must be focusable and operable via keyboard - Visible focus indicators on all interactive elements - Logical tab order through the form

Screen reader support: - Add `aria-label` or `aria-labelledby` to rating dot buttons (e.g., "Rate First Impression 3 out of 5") - Add `role="radiogroup"` to dot rating rows - Add `aria-selected` to archetype pills, reviewer type buttons - Add `aria-live` regions for the results panel so score changes are announced - Ensure case study add/remove actions are announced

Touch targets: - All interactive elements should have a minimum 44x44px touch target (WCAG 2.5.5) - Current rating dots are 28x28px (main) and 24x24px (case study) – both below minimum - Buttons and pills should be reviewed for touch target size

Motion & animation: - Respect `prefers-reduced-motion` media query for theme transitions, dot animations, and progress ring - Add `@media (prefers-reduced-motion: reduce)` block disabling transitions

Text scaling: - Ensure the layout doesn't break at 200% browser zoom - Use relative units (`rem`) instead of `px` where possible to respect user font size preferences

Implementation approach

1. **Phase 1 (immediate):** Apply the tiered type scale – bump the worst offenders (7-9px) and establish the 12px floor. This is the quick win that addresses the direct user feedback.
2. **Phase 2 (follow-up):** Full WCAG 2.1 AA audit covering contrast, keyboard, screen reader, touch targets, and motion. This is a larger effort but critical given the scorecard evaluates *accessibility as a dimension itself* (the "Responsiveness & A11y" shell dimension).

> ***Note:*** *A portfolio evaluation tool that itself fails accessibility standards undermines its credibility when rating others on accessibility.*

REQ-16: Rename “Case Studies” -> “Project / Case Study Assessment”

Status: Ready to build **Priority:** – **Section affected:** Section 02, results panel, pattern descriptions, guidance labels, growth areas

Requirement

Rename the section from “Case Studies” to “Project / Case Study Assessment” throughout the tool. Not all portfolio work is structured as a traditional case study – some are shipped products, side projects, or design explorations. The broader framing accommodates all of these.

All references to update

Section header:

Current	Proposed
`Case Studies`	`Project / Case Study`
`Rate each case study individually -- add up to 6`	`Rate each project or case study individually -- add up to 6`

Input UI:

Current	Proposed
`Case study ${n} name...` (placeholder)	`Project ${n} name...`
`+ Add Case Study`	`+ Add Project`
`(max 6)`	`(max 6)` – no change

Results panel:

Current	Proposed
`Case Study Ranking`	`Project Ranking`
`Case Studies` (stat label)	`Projects`
`Case Studies` (growth area section label)	`Projects`
`Case study emphasis` (guidance label)	`Project emphasis`

Pattern descriptions:

Current	Proposed
`"...but case studies lack depth."`	`"...but projects lack depth."`
`"Strong case studies hidden behind..."`	`"Strong projects hidden behind..."`

Page subtitle:

Current	Proposed
`"...the site, the case studies, and the strategy..."`	`"...the site, the projects, and the strategy..."`

Engine/code references (internal – less visible but keep consistent):

Variable names like `caseStudies`, `CS_DIMS`, `csScores`, etc. can stay unchanged to avoid a large refactor – these are internal and don’t affect what users see. The rename is a skin/copy concern, not an engine restructure.

REQ-13: Portfolio Purpose Framework & Leadership Dimensions

Status: Brainstorming – needs exploration before build **Priority:** – **Type:** Strategic / structural – may reshape existing dimensions or become a separate tool

> ***[!] Implementation note:*** *This REQ should always remain the last item in the backlog. It is an exploratory / “play” layer that builds on top of the scorecard. Any time the base scorecard changes (new dimensions, restructured sections, scoring changes, etc.), a* ***scorecard version bump*** *is required before this REQ can be revisited. This ensures the foundation is stable before layering analytical frameworks on top of it.*

Raw meeting notes

Leadership skills in portfolios: - Being personable – storytelling, branding, awareness of communication, distilling complex projects into simplicity - “The work you do speaks for you”

What portfolios are for (three layers):

1. **Communicate** – what you know, and prove you’re good at it
2. **Communicate** – what you do, who you are, what you’re about, what projects you’ve worked on – and you want to be wowed
3. **Convey** – how you work, how you problem solve

Key assertion: It should not be cookie cutter. These are all measurable things.

Analysis: What’s being described

These notes are articulating a *philosophy of portfolio evaluation* – the “why” behind the dimensions we’re already scoring. They break down into three distinct evaluation lenses:

Lens	Question it answers	What it measures
Competence	“Are they good?”	Knowledge, skill depth, craft quality
Identity	“Who are they?”	Personality, brand, positioning, wow factor
Process	“How do they think?”	Problem-solving, methodology, leadership in complexity

The current scorecard *partially* covers these across its three sections, but the mapping is implicit rather than explicit:

Current dimension	Lens it serves
First Impression	Identity
About & Personal Brand	Identity
Storytelling (REQ-11)	Identity + Process
Visual Design Quality	Competence
Communication: Writing & Copy (REQ-11)	Competence + Identity
Problem Framing (case study)	Process
Process & Methodology (case study)	Process
Outcomes & Impact (case study)	Competence
Differentiation (strategy)	Identity

The leadership angle – being personable, distilling complexity, work speaking for itself – is not currently captured as a standalone dimension anywhere. It’s a meta-skill: can this designer *lead through their portfolio*? Can they make you trust their judgment before you’ve ever met them?

Three paths forward

Path A: Integrate into the existing scorecard

Add these as new dimensions or rename existing ones to make the three lenses explicit. No new tool required.

Where it could fit without structural changes:

Concept	Proposed integration
“Prove you’re good at it”	Already covered by Outcomes & Impact, Visual Execution
“Who you are / what you’re about”	Already covered by About & Personal Brand
“Want to be wowed”	Already covered by First Impression, Differentiation
“How you problem solve”	Already covered by Problem Framing, Process & Methodology
“Not cookie cutter”	Already covered by Differentiation
“Distilling complexity”	**Gap** – could add as a new shell or strategy dimension: “Clarity of Communication” or fold into Communication: Writing & Copy
“Personable / leadership presence”	**Gap** – could add as a new shell dimension: “Leadership Presence” or “Designer Voice”
“Work speaks for you”	**Gap** – this is a meta-quality about self-evidence. Could become a strategy dimension: “Self-Evidence” – does the portfolio prove competence without needing to be explained?

Pros: Low effort, keeps everything in one tool. **Cons:** The scorecard is already growing (9 shell dims + Storytelling). Adding more risks dimension fatigue.

Path B: Separate “Portfolio Philosophy” evaluation tool

A standalone companion document or tool that asks higher-level questions. Less granular than the scorecard – more like a rubric or checklist that a reviewer fills out in prose. Think of it as the *qualitative* counterpart to the scorecard’s *quantitative* ratings.

Structure could follow the three lenses directly:

```
Section 1: Competence -- Does this portfolio prove they're good?
Section 2: Identity -- Do you know who this designer is?
Section 3: Process -- Do you understand how they think?
Section 4: Leadership -- Would you trust this person to lead?
```

Each section has 2-3 guided prompts with open text responses rather than 1-5 dot ratings.

Pros: Clean separation of concerns. Quantitative scores stay in the scorecard, qualitative evaluation lives here. Can be used independently or alongside the scorecard. **Cons:** Another tool to maintain. Users might not want to fill out both.

Path C: Add a “Portfolio Philosophy” overlay to the existing scorecard

The three lenses become a *secondary grouping* on the results panel. The dimensions stay the same, but the results view can toggle between “by section” (Shell / Case Studies / Strategy) and “by lens” (Competence / Identity / Process). This reframes the same scores through a different analytical angle without adding any new input fields.

```
const LENSES = {
  competence: {
    label: 'Competence',
    desc: 'Does this portfolio prove they are good at what they do?',
    dimensions: ['visual_quality', 'outcomes', 'visual_execution', 'writing_quality'],
  },
  identity: {
    label: 'Identity',
    desc: 'Do you know who this designer is and what they stand for?',
    dimensions: ['first_impression', 'about_story', 'storytelling', 'differentiation', 'contact_cta'],
  },
  process: {
    label: 'Process',
    desc: 'Do you understand how they think and solve problems?',
    dimensions: ['problem_framing', 'process_depth', 'role_clarity', 'navigation'],
  },
};
```

Pros: Zero new input fields. Reuses existing data. Adds analytical depth to the results panel. **Cons:** Some dimensions don’t map cleanly to one lens. Could feel forced.

Recommendation

Start with Path A (small) + Path C (results view). Add 1-2 missing dimensions to cover the gaps (leadership presence, self-evidence), then add the lens-based results view as an alternative analysis mode. This delivers the most value with the least structural change. If the concept proves useful in workshops, Path B (standalone tool) becomes a follow-up project.

Open questions

#	Question	Notes
1	Is this for the portfolio scorecard, a separate tool, or both?	Path A, B, or C above
2	Should “leadership presence” be a shell dimension, a strategy dimension, or something else?	Shell = evaluating the site’s tone. Strategy = evaluating positioning choices.
3	How important is the “not cookie cutter” signal?	Differentiation partially covers this, but it might warrant stronger language or a more prominent placement.
4	Should the three lenses (Competence / Identity / Process) be visible to the user filling out the scorecard, or only in the results analysis?	Showing them during input could guide better ratings. Showing only in results keeps the input simple.